Cómo implementar Dobble en JavaScript para web

Sí: la forma más práctica de implementar Dobble en JavaScript para web es generar primero el mazo como una estructura combinatoria basada en un **plano proyectivo finito** y luego renderizar cada carta con HTML/CSS/SVG o canvas.[1] [2] [3]

Estructura del algoritmo

Para un Dobble de orden n, cada carta tiene $n+1$ símbolos y el mazo completo tiene n^2+n+1 cartas y el mismo número de símbolos totales.[4] [5] [6]
La construcción más simple para código usa aritmética modular cuando n es primo, lo que permite generar cartas donde cualquier par comparte exactamente un único símbolo.[2] [3] [1]

JavaScript base

Este generador devuelve un mazo válido como arrays de IDs numéricos; para Dobble clásico, usa `n = 7`, lo que produce 57 cartas de 8 símbolos cada una.[6] [7] [3]

```
function generateDobbleDeck(n) {
  const cards = [];

  // Carta inicial
  const firstCard = [];
  for (let i = 0; i <= n; i++) firstCard.push(i);
  cards.push(firstCard);

  // Siguientes n cartas
  for (let j = 0; j < n; j++) {
    const card = [^0];
    for (let k = 0; k < n; k++) {
      card.push(n + 1 + n * j + k + 1);
    }
    cards.push(card);
  }

  // Últimas n^2 cartas
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      const card = [i + 1];
      for (let k = 0; k < n; k++) {
        const value = n + 2 + n * k + ((i * k + j) % n);
        card.push(value);
      }
      cards.push(card);
```

```
    }
  }

  return cards;
}
```

Ese patrón coincide con implementaciones conocidas del problema y con la construcción modular usada para mazos tipo Spot It!/Dobble cuando el orden es primo.[3] [1] [2]

Validación

Antes de dibujar cartas en pantalla, conviene validar que toda pareja de cartas tenga intersección de tamaño 1.[8] [6]
Una validación simple sería esta:

```
function validateDeck(deck) {
  for (let i = 0; i < deck.length; i++) {
    for (let j = i + 1; j < deck.length; j++) {
      const a = new Set(deck[i]);
      const common = deck[j].filter(x => a.has(x));
      if (common.length !== 1) {
        return { valid: false, pair: [i, j], common };
      }
    }
  }
  return { valid: true };
}
```

Esa comprobación te protege de errores al adaptar el algoritmo, cambiar el orden, o recortar cartas para una versión comercial.[6] [8]

Uso en una web

En la app web, lo ideal es separar tres capas: `generateDobbleDeck(n)`, un mapeo `symbolId -> asset`, y un renderer visual.[9] [3]
Por ejemplo, puedes tener un array como `symbols = [{id:0, img:"/icons/apple.svg"}, ...]` y luego convertir cada carta en un componente React o en un bloque HTML con posiciones aleatorias controladas.[9] [3]

Ejemplo de render simple:

```
const deck = generateDobbleDeck(7);
const symbols = Array.from({ length: 57 }, (_, id) => ({
  id,
  label: `symbol-${id}`,
  img: `/assets/${id}.png`
}));

function getCardData(card) {
```

```
  return card.map(id => symbols.find(s => s.id === id));
}
```

Recomendaciones de implementación

- Usa `n = 2, 3 o 5` para pruebas rápidas, porque generan mazos pequeños y fáciles de inspeccionar.[9] [6]
- Mantén el motor matemático desacoplado del diseño visual, así podrás cambiar íconos, temas o exportación sin tocar la lógica.[3] [9]
- Si quieres un mazo "tipo comercial", genera el mazo completo y luego elimina algunas cartas, pero vuelve a validar el resultado final.[7] [10]
- Para impresión, SVG suele ser mejor que canvas porque escala sin perder calidad.[11]

Si te sirve, el siguiente paso natural es que te pase una versión lista para usar en **TypeScript + React/Next.js**, con generador, validador y componente visual de carta.

⁂

1. https://joelgrus.com/2015/06/12/on-the-mathematics-of-spot-it/
2. https://liorsinai.github.io/mathematics/2023/04/09/dobble.html
3. https://ichi.pro/fr/dobble-theorie-et-mise-en-oeuvre-280520919044888
4. https://math.mit.edu/research/highschool/primes/materials/2024/May/6-5 Katherine.pdf
5. https://www.apmep.fr/IMG/pdf/10-LambreV_C.pdf
6. https://www.hicreategames.com/the-math-behind-spot-it/
7. https://arxiv.org/abs/1301.7058
8. https://openprairie.sdstate.edu/cgi/viewcontent.cgi?article=1016&context=jur
9. https://www.pairpops.com/post/the-math-behind-matching-games-like-spot-it-dobble
10. https://elpais.com/ciencia/cafe-y-teoremas/2023-10-25/dobble-el-juego-de-mesa-que-esconde-geometrias-avanzadas.html
11. https://cdn.svc.asmodee.net/production-asmodeees/uploads/2023/06/DobbleCollector_ES_Rules.pdf
12. https://blog.plover.com/math/finite-projective-planes.html
13. https://stackoverflow.com/questions/6240113/what-are-the-mathematical-computational-principles-behind-this-game
14. https://www.youtube.com/watch?v=-g3r-uvDey0
15. https://www.reddit.com/r/mathriddles/comments/2wapg8/the_mathematics_of_spot_it_cards/
16. https://hapax.github.io/mathematics/spotit/
17. https://cpge-paradise.com/TIPE/albdelkayoum_kaddouri/PPT_kaddouri.pdf